In section 24.2 I show how authorization fits into an ASP.NET Core web application and how it relates to the `ClaimsPrincipal` class you saw in the previous chapter. You'll see how to enforce the simplest level of authorization in an ASP.NET Core app, ensuring that only authenticated users can execute a Razor Page or MVC action. This chapter focuses on authorization in Razor Pages and Model-View-Controller (MVC) controllers; in chapter 25 you'll learn how the same principles apply to minimal API applications.

We'll extend that approach in section 24.3 by adding the concept of policies. These let you set specific requirements for a given authenticated user, requiring that they have specific pieces of information to execute an action or Razor Page.

You'll use policies extensively in the ASP.NET Core authorization system, so in section 24.4 we'll explore how to handle more complex scenarios. You'll learn about authorization requirements and handlers, and how you can combine them to create specific policies that you can apply to your Razor Pages and actions.

Sometimes whether a user is authorized depends on which resource or document they're attempting to access. A resource is anything that you're trying to protect, so it could be a document or a post in a social media app. For example, you may allow users to create documents or to read documents from other users, but to edit only documents that they created themselves. This type of authorization, where you need the details of the document to determine if the user is authorized, is called *resource-based authorization*, and it's the focus of section 24.5.

In the final section of this chapter I show how you can extend the resource-based authorization approach to your Razor view templates. This lets you modify the UI to hide elements that users aren't authorized to interact with.